

Ingestion Pipeline as a Contract

Topic 1 of 20 - Controlled text source for retrieval evaluation

A retrieval system begins before search. The ingestion pipeline decides what the system will later be able to retrieve. A useful way to think about ingestion is as a contract between the source document and the search index. The contract states that each meaningful piece of source content must be represented as clean text, attached to reliable metadata, and stored in a form that can be searched repeatedly. If ingestion silently drops a paragraph, the search layer cannot recover it. If ingestion duplicates headers on every page, the index may overrepresent those repeated words and distort ranking. Good retrieval therefore depends on disciplined source processing rather than on the search algorithm alone.

A production ingestion flow often has several stages. It identifies the file type, reads the document, extracts text, normalizes the result, creates chunks, computes embeddings, and stores both the text and metadata. Each stage should have a clear input and output. This makes failures observable. For example, the extraction stage should report how many pages produced text, while the chunking stage should report the number of chunks and their page ranges. A large drop between extracted items and stored chunks may indicate aggressive filtering, accidental deduplication, or an unexpected parser behavior. These checks are simple, but they prevent later search failures from being blamed on the wrong component.

The most important design rule is to preserve provenance. Every chunk should retain enough information to trace it back to its source, such as file identifier, page range, and chunk index. Provenance lets evaluators ask not only whether a result sounds relevant, but whether it came from the correct location. It also supports debugging when two similar passages compete in retrieval. In a controlled benchmark, provenance becomes part of the ground truth: the evaluator can compare the returned page range with the expected page and then inspect the retrieved text for distinctive evidence. This turns a vague search test into a repeatable engineering experiment.

Text Layers, OCR, and Source Selection

Topic 2 of 20 - Controlled text source for retrieval evaluation

Documents that look identical on screen can behave very differently during extraction. A PDF may contain a real text layer, a scanned image of text, or a mixture of both. A text-layer PDF stores characters directly, so a parser can usually recover sentences with high accuracy. A scanned PDF stores pixels instead of characters, so an OCR engine must first detect text regions and recognize the letters. Mixed documents are common in lecture notes because typed paragraphs, screenshots, diagrams, and scanned pages may appear together. The ingestion system should identify which representation is available instead of applying the same extraction method to every page.

OCR is valuable when text exists only inside images, but OCR is not the same as document understanding. It can recognize labels such as 'pooling layer', 'input', or 'output', yet it may not understand that an arrow connects one object to another. A diagram can therefore contain important knowledge that is only partially preserved by OCR. This matters for retrieval evaluation. If a benchmark question requires interpreting a diagram relationship, a text-only search system may appear to fail even though the search algorithm is working correctly. The benchmark should match the information that the production ingestion pipeline is actually designed to preserve.

For a controlled chunking experiment, a text-based source is usually preferable. The goal is to compare chunk sizes, not to measure OCR accuracy at the same time. Using a document with a reliable text layer removes one major source of variation. OCR can remain part of the production pipeline for real-world files, but the benchmark document should isolate the variable under study. This separation of concerns makes the result easier to explain: if 500-token chunks outperform 350-token chunks, the difference is more likely to come from chunk boundaries and context size rather than from inconsistent character recognition.

Normalization Without Semantic Damage

Topic 3 of 20 - Controlled text source for retrieval evaluation

Extracted text is rarely ready for indexing exactly as it appears. Parsers may produce repeated spaces, unusual line breaks, page numbers, headers, footers, or broken words. Normalization cleans this representation so that semantically identical text is treated consistently. A safe normalization step can collapse repeated whitespace, standardize common punctuation, and remove repeated template text that appears on every page. The objective is not to rewrite the document. It is to remove formatting noise while preserving the words that carry meaning.

Over-normalization can be harmful. Converting everything to plain ASCII may destroy technical symbols, while removing all punctuation can merge concepts that were originally separate. Deleting short lines indiscriminately may remove headings, labels, or concise definitions. Even lowercasing should be used with awareness: it is helpful for case-insensitive matching, but the stored source text may still need its original capitalization for display. A practical pipeline often stores clean display text and applies additional normalization only inside search functions. That approach keeps retrieval flexible without losing the original wording.

A benchmark should also avoid accidental normalization shortcuts. If the evaluation evidence is copied from a source sentence but the pipeline inserts line breaks or changes dash characters, a literal substring check may fail even when the correct chunk was retrieved. The evaluator can reduce this problem by applying the same lightweight normalization to both chunk content and acceptable evidence. Collapsing whitespace and using case-insensitive comparison are usually safe. The benchmark should still use short, distinctive evidence phrases rather than long exact sentences, because long phrases are more vulnerable to harmless formatting changes.

Normalization should also be deterministic. The same source file processed twice should produce the same normalized text unless the software version or configuration changes. Determinism matters because chunk boundaries depend on the exact character stream. Even a small difference in inserted spaces or removed headers can shift later token counts and change many chunks. Recording the normalization version alongside evaluation results makes later comparisons more trustworthy.

Why Chunk Boundaries Matter

Topic 4 of 20 - Controlled text source for retrieval evaluation

Chunking transforms a continuous document into retrieval units. Search does not normally return an entire book or an entire lecture note; it returns smaller passages. The location of each chunk boundary therefore changes what information can be retrieved together. A boundary placed between a problem statement and its explanation may separate two ideas that belong together. A larger chunk may preserve the relationship, while a smaller chunk may retrieve only one half. This is why chunk size is not merely a storage setting. It changes the semantic units available to the search engine.

Consider a page that first describes a failure mode and then, in the next paragraph, explains the mitigation. A user may ask why the mitigation is necessary. To answer well, the retrieved context ideally contains both the failure and the corrective action. If a 350-token chunk ends after the failure paragraph and the next chunk begins with the mitigation, the retrieval system must rank the second chunk correctly or return both. A 500-token chunk may keep the two paragraphs together and make the relationship easier to retrieve. The opposite can also occur: a larger chunk may dilute a precise signal with unrelated neighboring text.

The effect of boundaries is best measured with questions whose answers require nearby context rather than isolated keywords. A benchmark that asks only for one short definition may produce identical results for several chunk sizes. Boundary-sensitive questions are more informative because they create realistic competition between precision and context preservation. For a fair comparison, the underlying extracted text, embeddings model, search configuration, and evaluation questions should remain unchanged. Only the chunking configuration should differ.

Boundary effects are easiest to observe when retrieval output is inspected alongside chunk metadata. Two chunks can originate from the same page yet differ in whether they include the sentence that connects a cause to an effect. Saving chunk indexes and short snippets for each returned result helps reviewers see these differences directly instead of inferring them from a single aggregate score.

Fixed-Size Chunking

Topic 5 of 20 - Controlled text source for retrieval evaluation

Fixed-size chunking is one of the simplest strategies for building retrieval units. The system accumulates text until it reaches a target token budget, creates a chunk, and then continues with the remaining text. The target may be 350 tokens, 500 tokens, or another value chosen for the application. Fixed-size chunking is attractive because it is predictable and easy to benchmark. It also makes index size roughly proportional to document length, which simplifies capacity planning.

The main weakness is that token boundaries do not understand document structure. A chunk may end in the middle of a topic transition or between two paragraphs that explain the same concept. Some implementations try to reduce this problem by preferring natural break points, such as paragraph ends, while still respecting the maximum token budget. This hybrid approach keeps chunk sizes controlled without cutting text arbitrarily. In evaluation, the exact implementation matters: two systems can both claim to use 350-token chunks while producing different boundaries because one respects paragraph structure and the other slices strictly by token count.

Setting	Expected effect
Smaller fixed size	More chunks; narrower context
Larger fixed size	Fewer chunks; broader context
Paragraph-aware split	Cleaner semantic boundaries

A fair benchmark should record more than the configured size. It should also record the number of chunks produced, the minimum and maximum page ranges, and whether the same source text was used for each configuration. These measurements help explain the result. A smaller chunk size usually produces more chunks, which increases retrieval competition. A larger size usually produces fewer chunks, but each chunk contains more unrelated words. Neither outcome is automatically better; the benchmark exists to measure the trade-off.

Token counts are model-dependent rather than equal to word counts. A technical identifier, punctuation sequence, or uncommon word may consume several tokens, while familiar words may be represented compactly. This means a 350-token target does not correspond to a fixed number of words or lines. Evaluators should therefore measure the chunks produced by the actual tokenizer used in production instead of estimating boundaries from visual page length alone.

Overlap and Context Preservation

Topic 6 of 20 - Controlled text source for retrieval evaluation

Chunk overlap repeats a small amount of text between neighboring chunks. Its purpose is to reduce the chance that a useful fact is split exactly at a boundary. For example, if one chunk ends with the phrase 'the system becomes unstable when' and the next chunk begins with the explanation, an overlap can place part of the surrounding context in both chunks. This makes either chunk more understandable when retrieved alone. Overlap is especially useful when the chunker cannot reliably detect semantic boundaries.

Overlap also has costs. Repeated text increases the number of indexed tokens and can create near-duplicate search results. If the top five results contain three overlapping versions of the same passage, retrieval diversity decreases. The user sees fewer distinct pieces of evidence even though the system technically returned five chunks. For this reason, overlap should be treated as a controlled parameter rather than a free improvement. Systems may also apply duplicate suppression or diversity ranking after retrieval to reduce repeated results.

When comparing 350-token and 500-token chunks, overlap should remain constant unless overlap itself is the variable being tested. Otherwise two settings change at once and the experiment becomes difficult to interpret. A useful evaluation report states the chunk size, overlap, embedding model, candidate limit, final limit, and any deduplication behavior. Reproducibility depends on these details. If a future developer reruns the same benchmark, they should be able to recreate the same chunk boundaries and search conditions.

An overlap value should be large enough to preserve short transitions but small enough to avoid excessive repetition. The correct value depends on the writing style of the corpus. Documents with short paragraphs may need little overlap, while dense prose with long explanations may benefit from more. In a controlled study, the important point is not to discover the perfect overlap at the same time as chunk size, but to keep overlap stable while chunk size changes.

Boundary-Sensitive Facts

Topic 7 of 20 - Controlled text source for retrieval evaluation

Some facts are self-contained, while others depend on neighboring statements. This distinction is important when designing retrieval questions. A self-contained fact might be a definition such as 'cosine similarity compares vector direction'. It can be answered from one sentence. A boundary-sensitive fact is different: the document may describe a problem in one paragraph and explain its consequence or remedy in the next paragraph. The useful evidence is spread across a local region even though it remains on the same page.

Boundary-sensitive questions are valuable because they reveal whether a chunk size preserves enough local context. Suppose one paragraph explains that aggressive deduplication can remove legitimate repeated terminology. The next paragraph states that the safer solution is to deduplicate only boilerplate that repeats across many pages. A question asking why boilerplate-aware deduplication is preferred requires the relationship between those two paragraphs. A chunk that contains only the recommendation but not the reason may still be partially relevant, yet it provides weaker context for downstream answer generation.

A controlled corpus should include several of these cases without making every question dependent on boundaries. If all questions are boundary-sensitive, the benchmark may unfairly reward large chunks. If none are boundary-sensitive, chunk size may barely matter. A balanced design can mix direct definitions, comparative questions, and a smaller set of local two-paragraph relationships. This creates realistic retrieval competition while keeping the ground truth understandable.

Ground truth for these questions should identify a small amount of distinctive evidence from the relationship, not just a generic word shared by both paragraphs. For example, evidence such as 'boilerplate that repeats across many pages' is more discriminative than the single word 'deduplication'. Distinctive evidence reduces false hits when nearby sections discuss the same broad topic using similar vocabulary.

Embeddings as Semantic Coordinates

Topic 8 of 20 - Controlled text source for retrieval evaluation

An embedding model converts text into a vector: a list of numbers that represents semantic features learned by the model. The vector is not a human-readable summary. Instead, it places related pieces of text near one another in a high-dimensional space. A question about database rollback may therefore be close to a paragraph about undoing an uncommitted transaction even if the exact words are different. This semantic matching is the main reason vector search can retrieve paraphrased information.

Embeddings are created for both stored chunks and incoming queries. The document vectors are usually computed during ingestion and saved in the database or vector index. The query vector is computed at search time. A similarity function then compares the query vector with candidate chunk vectors. The embedding model used for documents and queries must be compatible. If document vectors come from one model and query vectors from an unrelated model, their coordinate spaces do not align and similarity scores become meaningless.

Embedding quality does not remove the need for good chunking. A vector represents the text it receives. If a chunk mixes three unrelated topics, its embedding may sit between them and become less precise for each one. If a chunk is too small, it may lose the context needed to express the intended meaning. The chunking experiment therefore evaluates how well the same embedding model represents different retrieval units. Keeping the model fixed isolates the effect of chunk size.

Vector dimensions should not be interpreted as named human concepts. A 1024-dimensional embedding does not mean that dimension 1 represents databases and dimension 2 represents chunking. Meaning is distributed across many values learned during training. For evaluation, the important properties are consistency and retrieval behavior: the same model should encode all document chunks and all questions using the intended query or passage mode.

Similarity Metrics and Vector Normalization

Topic 9 of 20 - Controlled text source for retrieval evaluation

Vector search requires a rule for deciding which vectors are close. Cosine similarity is common because it focuses on the angle between vectors rather than their raw magnitude. Two vectors that point in nearly the same direction receive a high cosine similarity even if one has larger numeric values. Dot product is another common metric. When vectors are normalized to unit length, dot product and cosine similarity become closely related, which can simplify ranking implementations.

The choice of metric should match the embedding model and index configuration. A system that normalizes document vectors but not query vectors may still work with cosine similarity, but its assumptions should be explicit. More importantly, the same metric must be used consistently during evaluation. Changing chunk size and similarity configuration at the same time makes it impossible to know which change caused an improvement. Controlled experiments keep the scoring rule constant.

Similarity scores are useful for ranking, but the numeric value alone is not proof of relevance. A top-ranked chunk can still be wrong if many passages discuss related vocabulary. This is common in technical documents where the same terms recur across several sections. Evaluation therefore compares returned chunks against ground truth pages and evidence. The ranking score explains why a candidate was chosen; the ground truth determines whether that choice was useful.

Indexes may report similarity or distance, and the direction of the score can differ. In one system a larger cosine score is better; in another a smaller distance is better. Evaluation code should not assume score direction without checking the index contract. Logging both the returned rank and raw score can reveal configuration mistakes, especially when apparently unrelated chunks receive unusually strong values.

Query and Document Embeddings

Topic 10 of 20 - Controlled text source for retrieval evaluation

Some embedding models use the same encoding function for queries and documents, while others provide separate query and passage instructions. The distinction matters because a question and an explanatory paragraph have different linguistic forms. A query may be short and interrogative, whereas a document chunk may be longer and declarative. Models trained for retrieval can learn to map these forms into a compatible space even when their wording differs substantially.

In code, it is common to precompute all evaluation query vectors before running the search loop. A dictionary can map each question identifier to its vector, allowing repeated experiments to reuse the same query embedding. This saves time when comparing multiple chunk configurations. The mapping also reduces accidental mismatch because each vector is retrieved by a stable question identifier rather than by list position. Reproducibility improves when the same questions always use the same encoded vectors.

Query preprocessing should be consistent across configurations. If one run lowercases or rewrites the question and another run embeds the original wording, the experiment is no longer testing only chunk size. The same principle applies to document preprocessing. A controlled benchmark fixes the query text, embedding model, normalization rules, and search configuration, then changes only the chunking variable.

Batching is another practical consideration. Encoding twenty evaluation questions one at a time may be slower than encoding them in a batch, but the resulting vectors should be equivalent for deterministic inference. Precomputing the questions before the retrieval loop keeps timing noise out of the search measurement. It also ensures that both chunk-size runs use identical query vectors rather than recomputing them under potentially different runtime conditions.

For this reason, evaluation logs should note the embedding model name and version. A future model upgrade can change rankings even when every other setting remains identical.

Lexical Search and Exact Terms

Topic 11 of 20 - Controlled text source for retrieval evaluation

Lexical search ranks documents using the actual words that appear in the query and the indexed text. It is particularly strong for exact names, identifiers, error messages, code symbols, and rare technical terminology. A query containing 'rollback' should strongly favor a chunk that also contains 'rollback'. In many search engines, term frequency and document frequency are combined so that uncommon words carry more weight than words that appear everywhere.

Lexical search can struggle with paraphrases. A user might ask how to 'undo temporary database changes' while the document uses the word 'rollback'. The meaning is similar, but the exact terms differ. This is where vector retrieval can help. Conversely, vector search may retrieve semantically related but technically incorrect passages when precise identifiers matter. The strengths of the two approaches are complementary rather than mutually exclusive.

A controlled corpus should include both easy exact-term queries and paraphrased queries. If every question repeats the source wording, lexical search can dominate and hide the value of embeddings. If every question is heavily paraphrased, the benchmark may underrepresent real cases where users paste exact errors or names. A balanced set helps reveal how hybrid retrieval behaves under different query styles.

Repeated template text can hurt lexical ranking because it increases the frequency of words that are unrelated to the actual topic. A university name, course code, or footer repeated on every page can become common noise. Removing such boilerplate before indexing improves the usefulness of term statistics and prevents search results from being influenced by formatting elements that carry little semantic value.

Exact-term behavior is especially useful for debugging because the expected keyword can be searched directly in stored chunks. If the term is absent, the problem occurred before ranking.

Vector Search and Semantic Recall

Topic 12 of 20 - Controlled text source for retrieval evaluation

Vector search focuses on meaning rather than exact token overlap. It is useful when users describe a concept in different words from the source. For instance, a document may say that 'larger chunks contain broader context' while a question asks which chunk size preserves more surrounding information. A well-trained embedding model can place these expressions close together even though the wording is not identical.

High semantic recall can introduce false positives. Technical topics often share broad concepts such as performance, context, ranking, and filtering. A vector search may therefore retrieve a passage that sounds related but does not contain the specific answer. Candidate limits matter here. Retrieving more candidates can improve the chance that the correct passage is included, but it also gives downstream ranking more work. The final top-k should reflect what the application can realistically send to the answer generator.

For chunk-size evaluation, vector retrieval should operate over comparable indexes. The 350-token index and the 500-token index should represent the same document and use the same embedding model. If one index contains extra OCR text or different normalization, the measured difference cannot be attributed cleanly to chunking. Semantic recall is meaningful only when the underlying corpus is controlled.

Approximate nearest-neighbor indexes may trade a small amount of exactness for substantial speed on large corpora. For a small controlled benchmark, exact or highly reliable search is preferable when available because it removes another source of randomness. The benchmark should primarily expose chunking behavior, not the recall characteristics of an approximate index configuration.

Keeping query vectors fixed across repeated runs also reduces unnecessary computation and makes the comparison easier to reproduce on slower development machines.

That stability is useful when search experiments are run repeatedly during development.

Hybrid Search and Score Fusion

Topic 13 of 20 - Controlled text source for retrieval evaluation

Hybrid search combines lexical and vector signals. One branch retrieves candidates because their words match the query, while another branch retrieves candidates because their embeddings are semantically similar. The system then fuses the rankings. This approach is useful in technical knowledge bases because users alternate between exact terms and natural paraphrases. A query containing a function name may benefit from lexical matching, while a conceptual question may benefit more from vector similarity.

Score fusion can be implemented in several ways. Weighted score fusion rescales lexical and vector scores and combines them using configured weights. Rank-based methods such as reciprocal rank fusion use candidate positions instead of raw scores, which avoids some problems caused by incompatible score ranges. The fusion strategy should remain fixed during a chunk-size experiment. Otherwise a change in ranking behavior may be incorrectly attributed to the chunk size.

Signal	Typical strength
Lexical	Exact terms and identifiers
Vector	Paraphrases and semantic similarity
Hybrid	Combines both candidate signals

Hybrid search also creates realistic competition. Two chunks may both contain the same keyword, but only one may explain the query's intended concept. Vector similarity can separate them. In another case, several passages may be semantically related, but only one contains a rare identifier that lexical search strongly recognizes. A good benchmark includes these ambiguous situations because they resemble real retrieval workloads.

Because the two branches can return overlapping candidates, fusion should deduplicate candidates by a stable chunk identifier before the final limit is applied. Otherwise one chunk may occupy multiple positions simply because it was found by both lexical and vector search. Correct deduplication preserves the contribution of both signals while ensuring that the final top five represent five distinct retrieval units.

Metadata Filtering Before Ranking

Topic 14 of 20 - Controlled text source for retrieval evaluation

Search relevance is not only about text similarity. Applications often restrict retrieval by course, user, file, folder, document status, or active ingestion run. These metadata filters decide which chunks are eligible before ranking begins. A perfect embedding cannot retrieve a chunk that the database query has filtered out. This is why an empty result set should first be investigated as a filtering problem rather than as an embedding problem.

Filtering bugs can be subtle during evaluation. An evaluator may insert temporary chunks into the database but forget to create related metadata that the production query requires. If hybrid search joins chunks to an active ingestion record, evaluation chunks without such a record may disappear from every search. The database can contain the rows, and the embedding counts can look correct, yet the search function still returns zero candidates. Logging the number of candidates before relevance scoring helps separate this failure from ranking failure.

A controlled benchmark should use the same metadata path as production wherever practical. If temporary evaluation rows are needed, they should satisfy the same foreign-key and status requirements as normal ingested data. Cleanup can be handled with a transaction that is rolled back after the experiment, provided the code under test does not commit independently. This allows realistic search behavior without leaving permanent benchmark data in the database.

Filter order can also affect performance. Applying restrictive metadata conditions in the database is usually more efficient than retrieving a large global candidate set and discarding most of it later. More importantly for correctness, the evaluation must use the same eligibility rules as the application. A benchmark that bypasses ownership, status, or active-run filters may produce optimistic results that cannot occur in production.

Reranking the Candidate Set

Topic 15 of 20 - Controlled text source for retrieval evaluation

Initial retrieval is optimized for speed. It may collect dozens of plausible candidates using lexical and vector indexes. Reranking applies a more expensive relevance model to a much smaller candidate set and produces the final order. A reranker can examine the query and candidate text together, which often captures fine-grained relationships that independent embeddings miss. The final top five may therefore differ substantially from the initial nearest-neighbor order.

Reranking cannot rescue a candidate that was never retrieved. If the correct chunk is absent from the initial candidate pool, the reranker has nothing to promote. This creates two separate evaluation questions: candidate recall and final ranking quality. A low final Hit@5 can result from weak first-stage retrieval or from poor reranking. Logging intermediate candidate identifiers helps diagnose which stage lost the relevant passage.

Chunk size affects reranking input as well. A 500-token chunk provides more context but also gives the reranker more text to evaluate. A 350-token chunk is more focused but may omit the relationship needed to judge relevance. Because the reranker sees the actual chunk text, chunk boundaries can change its decisions even when the underlying source page is the same.

A useful diagnostic stores both the pre-rerank and post-rerank positions of the expected passage. If a relevant chunk appears at candidate rank three but falls to final rank twenty, reranking is the problem. If it never appears in the candidate set, first-stage retrieval is the problem. Separating these stages prevents developers from tuning the wrong component.

This stage-by-stage view is more informative than changing reranker weights immediately after seeing a low final score, because it identifies where relevance was actually lost.

It also makes regressions easier to locate in code review.

Duplicate Control and Diversity

Topic 16 of 20 - Controlled text source for retrieval evaluation

Retrieval systems can return near-duplicate chunks for several reasons. Overlap repeats neighboring text, repeated headers appear on many pages, and different chunk configurations may contain similar passages. If the top five results are dominated by duplicates, the system wastes retrieval slots. A user benefits more from five distinct pieces of evidence than from five versions of the same paragraph.

Duplicate control can occur before or after ranking. Exact duplicate hashes can remove identical text, while similarity-based methods can suppress near-duplicates. Diversity algorithms can also prefer results that are individually relevant but not redundant with results already selected. These techniques should be used carefully because repeated terminology may be legitimate. The goal is not to remove every repeated phrase; it is to prevent boilerplate and overlapping chunks from dominating the final result list.

In a chunk-size benchmark, duplicate behavior should be documented. Smaller chunks with overlap may create more near-duplicates than larger chunks. If the production search includes deduplication, the benchmark should keep that behavior enabled for both configurations. If deduplication is disabled for the experiment, the report should state that clearly because the observed Hit@5 may differ from production behavior.

Repeated headings deserve special attention because they are not always exact duplicates of a full chunk. A heading may be attached to different body text on several pages, making the chunks partially similar rather than identical. Diversity logic should therefore consider both lexical overlap and semantic similarity. The benchmark can reveal whether repeated structural text crowds out distinct evidence in the final results.

A simple report can count unique page ranges in the final top five. Very low diversity is a warning that overlap or boilerplate is consuming retrieval capacity.

Designing a Retrieval Benchmark

Topic 17 of 20 - Controlled text source for retrieval evaluation

A useful benchmark begins with a controlled corpus and a fixed set of questions. Each question needs ground truth that can be checked automatically. For a single PDF, file identity is not enough because every chunk has the same file identifier. Page ranges provide a stronger location signal. Short acceptable-evidence phrases provide an additional content check. A returned chunk can be considered relevant when its page range covers an expected page and its text contains at least one approved evidence phrase.

Question design should avoid trivial copying. If the question repeats an entire sentence from the source, lexical matching may make the task unrealistically easy. Paraphrased questions better represent how students and users ask for information. At the same time, the question must remain answerable from a clear source location. Ambiguous questions that have equally valid answers on many pages weaken the benchmark because a correct retrieval may be marked wrong by an overly narrow ground truth.

Coverage matters as much as question count. Questions should be spread across the beginning, middle, and end of the document. They should include direct facts, comparisons, and several boundary-sensitive relationships. Twenty carefully designed questions can be more informative than hundreds of automatically generated questions if each one has verified evidence and a clear purpose.

The benchmark file itself should be versioned or hashed so that results can be tied to an exact corpus. A small wording edit may move chunk boundaries and invalidate direct comparison with earlier runs. The question dataset should also have stable identifiers such as q01 through q20. Stable corpus and question identities make evaluation output easy to diff in code review and continuous integration.

The PDF should contain only original, non-sensitive material so it can be committed with the evaluation code and rerun by other developers without access restrictions.

Hit@K, Recall@K, and MRR

Topic 18 of 20 - Controlled text source for retrieval evaluation

Retrieval metrics answer different questions. Hit@K asks whether at least one relevant result appears in the top K. For each query, the outcome is binary: hit or miss. If 17 of 20 questions retrieve at least one relevant chunk in the top five, Hit@5 is 85 percent. This metric is intuitive when the application needs one useful passage to support an answer.

Recall@K is different when a query has multiple relevant items. It measures how many of all relevant items were retrieved in the top K. If four chunks are considered relevant and two appear in the top five, Recall@5 for that query is 50 percent. Mean reciprocal rank focuses on how early the first relevant result appears. A correct result at rank one receives more credit than the same result at rank five. These metrics describe different aspects of retrieval quality and should not be used interchangeably.

Metric	Main question
Hit@K	Did at least one relevant result appear?
Recall@K	How many relevant items were recovered?
MRR	How early was the first relevant result?

For a single-answer controlled benchmark, Hit@5 is often the simplest primary metric, while rank information can be retained for deeper analysis. The report can also record the first relevant rank and the top returned page numbers. This makes the final score easier to debug. A metric should summarize the run, but it should not hide the evidence needed to explain failures.

Aggregate scores should be paired with per-question results. Two configurations can achieve the same Hit@5 while succeeding on different questions. That difference may reveal a meaningful trade-off: one setting may perform better on exact definitions while another performs better on boundary-sensitive relationships. Per-question output preserves this information and makes a tied aggregate score easier to interpret.

Diagnosing Misses Systematically

Topic 19 of 20 - Controlled text source for retrieval evaluation

A retrieval miss can originate from several layers. The source text may never have been extracted. The text may have been extracted but split poorly. The chunk may exist in the database but be excluded by metadata filters. The correct chunk may enter the candidate set but receive a low score. Finally, the correct chunk may be returned but the evaluation function may fail to recognize it because of strict evidence matching. Treating every miss as an embedding failure leads to wasted debugging effort.

A practical diagnostic sequence starts with observability. First confirm that the expected page produced extracted text. Next confirm that a chunk covering that page exists. Then confirm that the chunk is stored with an embedding and valid metadata. Run the search and print the number of candidates returned. If there are no candidates, investigate filters and joins. If candidates exist, inspect their page ranges and snippets. If the correct chunk is present but marked as a miss, investigate the ground-truth matcher rather than the search algorithm.

Miss analysis should be saved as structured output whenever possible. For each question, record the status, returned page ranges, first relevant rank, and a short reason for failure. This turns evaluation into an engineering tool instead of a single percentage. When 350-token and 500-token runs differ, the diagnostic record can show whether the change came from boundary preservation, ranking competition, or duplicate behavior.

A small number of debug prints can save considerable time during early development. Printing the number of results, page ranges, and the first hundred characters of each top result immediately shows whether search returned nothing, returned the wrong region, or returned the right region with a failed evidence check. Once the pipeline is stable, the same information can be written to a structured JSON report instead of verbose console output.

Reproducible Comparison of 350 vs 500 Tokens

Topic 20 of 20 - Controlled text source for retrieval evaluation

A valid comparison changes one major variable at a time. For a 350-versus-500-token experiment, both runs should use the same PDF, extracted text, normalization rules, embedding model, query vectors, search configuration, metadata filters, candidate limits, and evaluation questions. Only the maximum chunk size should change. If the 500-token run also uses different OCR output or a different reranker, the result cannot be attributed confidently to chunk size.

The cleanest implementation is to evaluate each chunk configuration separately. Insert the 350-token chunks, run all questions, save the results, and remove or roll back those temporary rows. Then repeat with 500-token chunks. This prevents the two chunk sets from competing in the same search index. If both configurations are stored simultaneously, the search query must include a reliable variant filter; otherwise a run labeled '350' may retrieve 500-token chunks and invalidate the comparison.

The final report should include more than the winning percentage. Record the number of chunks produced by each configuration, Hit@5, first relevant ranks, and the questions whose outcomes differ. Those differing questions are the most valuable evidence because they reveal what the chunk size changed. A good experiment does not try to force one setting to win. It creates a stable environment in which the retrieval behavior can be observed, repeated, and explained.

When results are reported, the experiment should avoid language that assumes the larger or smaller chunk size is inherently superior. The correct choice depends on the corpus and retrieval workload. The benchmark provides evidence for this specific production pipeline. Future changes to the embedding model, reranker, overlap, or document style may justify rerunning the experiment rather than treating one historical result as permanent.